```
#ifndef GNSSRMAMII_H
#define GNSSRMAMII_H
#include <QtCore/QVariant>
#include <QtWidgets/QApplication>
#include <QtWidgets/QCheckBox>
#include <QtWidgets/QDialog>
#include <QtWidgets/QFrame>
#include <QtWidgets/QLabel>
#include <QtWidgets/QLineEdit>
#include <QtWidgets/QListView>
#include <QtWidgets/QPushButton>
#include <QtWidgets/QTabWidget>
#include <QtWidgets/QTextBrowser>
#include <QtWidgets/QWidget>
QT_BEGIN_NAMESPACE
class Ui_Dialog
{
public:
    QPushButton *pushButton;
    QTabWidget *tabWidget;
    QWidget *tab;
    QCheckBox *checkBox;
    QCheckBox *checkBox_2;
    QFrame *line_2;
    QListView *listView_2;
    QTextBrowser *textBrowser_3;
    QWidget *tab_2;
    QCheckBox *checkBox_3;
    QCheckBox *checkBox_4;
    QFrame *line_3;
    QTextBrowser *textBrowser_2;
    QListView *listView;
    QFrame *line;
    QTextBrowser *textBrowser;
    QLabel *label;
    QLineEdit *lineEdit;
    QPushButton *pushButton_2;
    QTextBrowser *textBrowser_4;
    void setupUi(QDialog *Dialog)
    {
        if (Dialog->objectName().isEmpty())
            Dialog->setObjectName(QString::fromUtf8("Dialog"));
        Dialog->resize(400, 300);
        pushButton = new QPushButton(Dialog);
        pushButton->setObjectName(QString::fromUtf8("pushButton"));
        pushButton->setGeometry(QRect(310, 10, 81, 31));
        pushButton->setAutoRepeatDelay(300);
        tabWidget = new QTabWidget(Dialog);
        tabWidget->setObjectName(QString::fromUtf8("tabWidget"));
        tabWidget->setGeometry(QRect(0, 50, 391, 251));
```

```
tab = new QWidget();
tab->setObjectName(QString::fromUtf8("tab"));
checkBox = new QCheckBox(tab);
checkBox->setObjectName(QString::fromUtf8("checkBox"));
checkBox->setGeometry(QRect(320, 10, 61, 23));
checkBox_2 = new QCheckBox(tab);
checkBox_2->setObjectName(QString::fromUtf8("checkBox_2"));
checkBox_2->setGeometry(QRect(0, 10, 61, 23));
line_2 = new QFrame(tab);
line_2->setObjectName(QString::fromUtf8("line_2"));
line_2->setGeometry(QRect(0, 30, 381, 16));
line_2->setFrameShape(QFrame::HLine);
line_2->setFrameShadow(QFrame::Sunken);
listView_2 = new QListView(tab);
listView_2->setObjectName(QString::fromUtf8("listView_2"));
listView_2->setGeometry(QRect(0, 70, 381, 141));
textBrowser_3 = new QTextBrowser(tab);
textBrowser_3->setObjectName(QString::fromUtf8("textBrowser_3"));
textBrowser_3->setGeometry(QRect(0, 40, 381, 31));
tabWidget->addTab(tab, QString());
tab_2 = new QWidget();
tab_2->setObjectName(QString::fromUtf8("tab_2"));
checkBox_3 = new QCheckBox(tab_2);
checkBox_3->setObjectName(QString::fromUtf8("checkBox_3"));
checkBox_3->setGeometry(QRect(0, 10, 61, 23));
checkBox_4 = new QCheckBox(tab_2);
checkBox_4->setObjectName(QString::fromUtf8("checkBox_4"));
checkBox_4->setGeometry(QRect(320, 10, 61, 23));
line_3 = new QFrame(tab_2);
line_3->setObjectName(QString::fromUtf8("line_3"));
line_3->setGeometry(QRect(0, 30, 381, 16));
line_3->setFrameShape(QFrame::HLine);
line_3->setFrameShadow(QFrame::Sunken);
textBrowser_2 = new QTextBrowser(tab_2);
textBrowser_2->setObjectName(QString::fromUtf8("textBrowser_2"));
textBrowser_2->setGeometry(QRect(0, 40, 381, 31));
listView = new QListView(tab_2);
listView->setObjectName(QString::fromUtf8("listView"));
listView->setGeometry(QRect(0, 70, 381, 141));
tabWidget->addTab(tab_2, QString());
line = new QFrame(Dialog);
line->setObjectName(QString::fromUtf8("line"));
line->setGeometry(QRect(0, 40, 391, 16));
line->setFrameShape(QFrame::HLine);
line->setFrameShadow(QFrame::Sunken);
textBrowser = new QTextBrowser(Dialog);
textBrowser->setObjectName(QString::fromUtf8("textBrowser"));
textBrowser->setGeometry(QRect(10, 10, 51, 31));
label = new QLabel(Dialog);
label->setObjectName(QString::fromUtf8("label"));
```

```
        label->setGeometry(QRect(80, 20, 41, 17));
        lineEdit = new QLineEdit(Dialog);
        lineEdit->setObjectName(QString::fromUtf8("lineEdit"));
        lineEdit->setGeometry(QRect(120, 10, 161, 31));
        pushButton_2 = new QPushButton(Dialog);
        pushButton_2->setObjectName(QString::fromUtf8("pushButton_2"));
        pushButton_2->setGeometry(QRect(290, 50, 89, 25));
        textBrowser_4 = new QTextBrowser(Dialog);
        textBrowser_4->setObjectName(QString::fromUtf8("textBrowser_4"));
        textBrowser_4->setGeometry(QRect(160, 80, 81, 31));
        retranslateUi(Dialog);
        QObject::connect(pushButton_2, SIGNAL(clicked(bool)), textBrowser_4,
SLOT(setHidden(bool)));
        QObject::connect(pushButton, SIGNAL(clicked(bool)), textBrowser_4, SLOT(setVisible(bool)));
        tabWidget->setCurrentIndex(1);
        QMetaObject::connectSlotsByName(Dialog);
    } // setupUi
    void retranslateUi(QDialog *Dialog)
    {
        Dialog->setWindowTitle(QApplication::translate("Dialog", "Dialog", nullptr));
        pushButton->setText(QApplication::translate("Dialog", "\345\220\257\345\212\250",
nullptr));
        checkBox->setText(QApplication::translate("Dialog", "\346\240\241\345\207\206", nullptr));
        checkBox_2->setText(QApplication::translate("Dialog", "\350\277\236\346\216\245",
nullptr));
        textBrowser_3->setHtml(QApplication::translate("Dialog", "<!DOCTYPE HTML PUBLIC \"-
//W3C//DTD HTML 4.0//EN\" \"http://www.w3.org/TR/REC-html40/strict.dtd\">\n"
"<html><head><meta name=\"qrichtext\" content=\"1\" /><style type=\"text/css\">\n"
"p, li { white-space: pre-wrap; }\n"
"</style></head><body style=\" font-family:'Ubuntu'; font-size:11pt; font-weight:400; font-
style:normal;\">\n"
"<p align=\"center\" style=\" margin-top:0px; margin-bottom:0px; margin-left:0px; margin-right:0px;
-qt-block-indent:0; text-indent:0px;\"><span style=\" font-
size:9pt;\">\344\270\262\345\217\243\346\225\260\346\215\256</span></p></body></html>",
nullptr));
        tabWidget->setTabText(tabWidget->indexOf(tab), QApplication::translate("Dialog",
"\344\274\240\346\204\237\345\231\2501", nullptr));
        checkBox_3->setText(QApplication::translate("Dialog", "\350\277\236\346\216\245",
nullptr));
        checkBox_4->setText(QApplication::translate("Dialog", "\346\240\241\345\207\206",
nullptr));
        textBrowser_2->setHtml(QApplication::translate("Dialog", "<!DOCTYPE HTML PUBLIC \"-
//W3C//DTD HTML 4.0//EN\" \"http://www.w3.org/TR/REC-html40/strict.dtd\">\n"
"<html><head><meta name=\"qrichtext\" content=\"1\" /><style type=\"text/css\">\n"
"p, li { white-space: pre-wrap; }\n"
"</style></head><body style=\" font-family:'Ubuntu'; font-size:11pt; font-weight:400; font-
style:normal;\">\n"
"<p align=\"center\" style=\" margin-top:0px; margin-bottom:0px; margin-left:0px; margin-right:0px;
-qt-block-indent:0; text-indent:0px;\"><span style=\" font-
size:9pt;\">\344\270\262\345\217\243\346\225\260\346\215\256</span></p></body></html>",
```

```
nullptr));
        tabWidget->setTabText(tabWidget->indexOf(tab_2), QApplication::translate("Dialog", "\344\274\240\346\204\237\345\231\2502", nullptr));
        textBrowser->setHtml(QApplication::translate("Dialog", "<!DOCTYPE HTML PUBLIC \"-//W3C//DTD HTML 4.0//EN\" \"http://www.w3.org/TR/REC-html40/strict.dtd\">\n"
"<html><head><meta name=\"qrichtext\" content=\"1\" /><style type=\"text/css\">\n"
"p, li { white-space: pre-wrap; }\n"
"</style></head><body style=\" font-family:'Ubuntu'; font-size:11pt; font-weight:400; font-style:normal;\">\n"
"<p style=\" margin-top:0px; margin-bottom:0px; margin-left:0px; margin-right:0px; -qt-block-indent:0; text-indent:0px;\"><span style=\" font-size:10pt;\">GNSS</span></p></body></html>", nullptr));
        label->setText(QApplication::translate("Dialog", "UTC\357\274\232", nullptr));
        pushButton_2->setText(QApplication::translate("Dialog", "\346\240\207\345\256\232", nullptr));
        textBrowser_4->setHtml(QApplication::translate("Dialog", "<!DOCTYPE HTML PUBLIC \"-//W3C//DTD HTML 4.0//EN\" \"http://www.w3.org/TR/REC-html40/strict.dtd\">\n"
"<html><head><meta name=\"qrichtext\" content=\"1\" /><style type=\"text/css\">\n"
"p, li { white-space: pre-wrap; }\n"
"</style></head><body style=\" font-family:'Ubuntu'; font-size:11pt; font-weight:400; font-style:normal;\">\n"
"<p style=\" margin-top:0px; margin-bottom:0px; margin-left:0px; margin-right:0px; -qt-block-indent:0; text-indent:0px;\"><span style=\" font-size:10pt;\">\346\240\207\345\256\232\346\210\220\345\212\237\357\274\201</span></p></body></html>", nullptr));
    } // retranslateUi
};
namespace Ui {
    class Dialog: public Ui_Dialog {};
} // namespace Ui
QT_END_NAMESPACE
#endif // GNSSRMAMII_H
#include "main.h"
#include "dma.h"
#include "tim.h"
#include "usart.h"
#include "gpio.h"
#include <stdbool.h>
#include <string.h>
#include <time.h>
#include <stdio.h>
uint8_t ss001=0;// 0.01s
int i=0;//蜂鸣器计数
bool PPS_isArrived=false;//判断 PPS 是否到达
bool isfirsttime=true;//pps 是否第一次来
int  getInitTime_flag=0;//初始时间获取标志位
#define IMU_BUFFER_SIZE 170 //这里应该很小
#define GPS_BUFFER_SIZE 165
#define COMBINED_BUFFER_SIZE 330
char imuBuffer[IMU_BUFFER_SIZE];
```

```
char gpsBuffer[GPS_BUFFER_SIZE];
char combinedBuffer[COMBINED_BUFFER_SIZE];
char local_time_ymdsms;//本地年月日时分秒
char fisttime;//
char current_time_integer;//当前时间整数秒——
char lastGpsData[180];   // 假设 GPS 数据最大长度为 180 字符
volatile uint8_t gpsDataReady = 0;
volatile uint8_t imuDataReady = 0;
extern uint8_t u1_buffer[85], u2_buffer[150];
extern uint8_t u1_rx_size, u2_rx_size;
volatile int isTxInProgress = 0;
extern DMA_HandleTypeDef hdma_usart1_rx;
extern DMA_HandleTypeDef hdma_usart1_tx;
extern DMA_HandleTypeDef hdma_usart2_rx;
extern DMA_HandleTypeDef hdma_usart2_tx;
volatile bool pps_flag = false;//pps 来临标志符
volatile int txComplete = 0; // 传输完成标志
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
        if(GPIO_Pin==PPS_Pin)
        {
            pps_flag = true; // 只设置标志位
        }
}
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if(htim->Instance == TIM2)
    {
        HAL_GPIO_TogglePin(IMUtrigger_GPIO_Port, IMUtrigger_Pin);//每 0.01s 触发一次 IMU 数据
——并计时
        ss001++;//进一次回调函数累加一次说明累加了 0.01s(10ms)
    }
}
void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart) {
    if (huart == &huart5) {
        isTxInProgress = 0; // 仅重置标志位
        txComplete = 1;
    }
}
int main(void)
{
  /* USER CODE BEGIN 1 */
    IMU_data data={0}; // 所有成员初始化为零
    uint8_t dataimu[500];
    char dataimu_time[256];
    char utcTime[14]; // 用于存储提取的 UTC 时间
    char lastUpdatedTime[14]; // 上一次更新后的时间
    uint8_t byteArray[sizeof(IMU_data)];
    char timestamp[50]; // 添加时间戳
    RingBuffer imuDataBuffer;   // 创建一个环形缓冲区实例
```

```
HAL_Init();
SystemClock_Config();
MX_GPIO_Init();
MX_DMA_Init();
MX_TIM2_Init();
MX_USART1_UART_Init();
MX_USART2_UART_Init();
MX_UART5_Init();
Check_LED();//板子依次亮起，闪烁，最后保持全亮
Check_BEED();//蜂鸣器响一声，代表计时器启动
HAL_UARTEx_ReceiveToIdle_DMA(&huart1, u1_buffer, sizeof(u1_buffer));//启动 GPS 串 DMA 接收
__HAL_DMA_DISABLE_IT(&hdma_usart1_rx, DMA_IT_HT);
HAL_UARTEx_ReceiveToIdle_DMA(&huart2, u2_buffer, sizeof(u2_buffer));//启动 imu 串 DMA 接收
__HAL_DMA_DISABLE_IT(&hdma_usart2_rx, DMA_IT_HT);//关闭过半中断
strcpy(gpsBuffer, "No GPS data yet");
RingBuffer_Init(&imuDataBuffer);
char dataToSend[STRING_LENGTH];
while (1)
{
        if (pps_flag)
         {
              pps_flag = false;
              HAL_GPIO_TogglePin(LED0_GPIO_Port, LED0_Pin);//PPS 来的话 led0 闪烁
              if (isfirsttime && getInitTime_flag==0)
              {
                   isfirsttime = false;
                   getInitTime_flag=1;
                   HAL_TIM_Base_Start_IT(&htim2);//第一次 PPS 来开启定期器
              }
              else {
                   ss001 = 0;
                   __HAL_TIM_SET_COUNTER(&htim2, 0);//每次 pps 重置定时器防止晶振计时累计
                   PPS_isArrived=true;
              }
         }
   if(getInitTime_flag !=1 && PPS_isArrived) //不是第一次来 pps 数据
   {
        PPS_isArrived = false; //  重置
        incrementTime(lastUpdatedTime, 1); //  上一次更新后的时间加 1 秒
         long timeDiff = compareTime(lastUpdatedTime,utcTime);
          if ((timeDiff) <1 && (timeDiff) > -1) //  检查时间差是否在 1 秒以内
          {
          }
          else //非 1s 内则使用新的 utc 时间，PPS 在信号丢失时候会消失
          {
               strcpy(lastUpdatedTime, utcTime); //  将 lastUpdatedTime 更新为当前的 UTC 时间
          }
   }
   if (u1_rx_size)//串口获取到 imu 数据
   {
```

```
u1_rx_size=0;
  if (u1_buffer[0] != 0xFA && u1_buffer[1] != 0xFF&&u1_buffer[2] != 0x36) {
      printf("Invalid preamble or bus identifier\n");
}
int idx = 4;
uint32_t raw;
if (u1_buffer[idx] != 0x20 || u1_buffer[idx+1] != 0x30) {
      printf("Invalid data type identifier for Euler angles\n");
}
idx += 2;
if (u1_buffer[idx] != 0x0C) {
      printf("Invalid length for Euler angles data\n");
}
idx++;
for (int i = 0; i < 3; ++i) {
      raw = (u1_buffer[idx + 4 * i] << 24) |
            (u1_buffer[idx + 1 + 4 * i] << 16) |
            (u1_buffer[idx + 2 + 4 * i] << 8) |
            u1_buffer[idx + 3 + 4 * i];
      float *angle = (float*)&raw;
      switch (i) {
          case 0: data.roll = *angle; break;
          case 1: data.pitch = *angle; break;
          case 2: data.yaw = *angle; break;
      }
}
idx += 12;
if (u1_buffer[idx] != 0x40 || u1_buffer[idx+1] != 0x20) {
      printf("Invalid data type identifier for acceleration\n");
}
idx += 2;
if (u1_buffer[idx] != 0x0C) {
      printf("Invalid length for acceleration data\n");
}
idx++;
for (int i = 0; i < 3; ++i) {
      raw = (u1_buffer[idx + 4 * i] << 24) |
            (u1_buffer[idx + 1 + 4 * i] << 16) |
            (u1_buffer[idx + 2 + 4 * i] << 8) |
            u1_buffer[idx + 3 + 4 * i];
      float *accel = (float*)&raw;
      switch (i) {
          case 0: data.ax = *accel; break;
          case 1: data.ay = *accel; break;
          case 2: data.az = *accel; break;
      }
}
idx += 12;
if (u1_buffer[idx] != 0x40 || u1_buffer[idx+1] != 0x30) {
      printf("Invalid data type identifier for acc_free\n");
```

```
}
idx += 2;
if (u1_buffer[idx] != 0x0C) {
    printf("Invalid length for acc_free data\n");
}
idx++;
for (int i = 0; i < 3; ++i) {
    raw = (u1_buffer[idx + 4 * i] << 24) |
          (u1_buffer[idx + 1 + 4 * i] << 16) |
          (u1_buffer[idx + 2 + 4 * i] << 8) |
          u1_buffer[idx + 3 + 4 * i];
    float *Acc_free = (float*)&raw;
    switch (i) {
        case 0: data.ax_free = *Acc_free; break;
        case 1: data.ay_free = *Acc_free; break;
        case 2: data.az_free = *Acc_free; break;
    }
}
idx += 12;
if (u1_buffer[idx] != 0x80 || u1_buffer[idx+1] != 0x20) {
    printf("Invalid data type identifier for gyro\n");
}
idx += 2;
if (u1_buffer[idx] != 0x0C) {
    printf("Invalid length for gyro data\n");
}
idx++;
for (int i = 0; i < 3; ++i) {
    raw = (u1_buffer[idx + 4 * i] << 24) |
          (u1_buffer[idx + 1 + 4 * i] << 16) |
          (u1_buffer[idx + 2 + 4 * i] << 8) |
          u1_buffer[idx + 3 + 4 * i];
    float *gyro = (float*)&raw;
    switch (i) {
        case 0: data.gx = *gyro; break;
        case 1: data.gy = *gyro; break;
        case 2: data.gz = *gyro; break;
    }
}
idx += 12;
float fractionalSeconds = ss001 * 0.01;
char fractionalStr[10];
snprintf(fractionalStr, sizeof(fractionalStr), "%.2f", fractionalSeconds);
char* decimalPart = fractionalStr + 2;
snprintf(timestamp, sizeof(timestamp), "%s.%s", lastUpdatedTime, decimalPart);
char imubuffer[102]; // 足够大以容纳所有数据的字符串
int len= snprintf(imuBuffer, IMU_BUFFER_SIZE, "%s,%f,%f,%f,%f,%f,%f,%f,%f,%f,%f,%f,%f,%f\r\n",
        timestamp, // 时间戳
            data.roll, data.pitch, data.yaw,
            data.ax, data.ay, data.az,
```

```
                    data.ax_free, data.ay_free, data.az_free,
                    data.gx, data.gy, data.gz);
        if (RingBuffer_Put(&imuDataBuffer, imuBuffer) == 0) {
        }
        if ((!isTxInProgress && RingBuffer_Get(&imuDataBuffer, dataToSend))|| txComplete) {
            isTxInProgress = 1;
            txComplete = 0;
            HAL_UART_Transmit_DMA(&huart5, (uint8_t*)dataToSend, strlen(dataToSend));
        }
     }
    else if(u2_rx_size)//GPS 数据来
    {
        if (RingBuffer_Put(&imuDataBuffer, (char*)u2_buffer) == 0) {
        }
        if ((!isTxInProgress && RingBuffer_Get(&imuDataBuffer, dataToSend))|| txComplete) {
            isTxInProgress = 1;
            txComplete = 0;
            HAL_UART_Transmit_DMA(&huart5, (uint8_t*)dataToSend, strlen(dataToSend));
        }
        memcpy(gpsBuffer, u2_buffer,u2_rx_size);
        u2_rx_size=0;
        extractUTC(gpsBuffer, utcTime);//提取其中的整数秒 UTC 时间
        if(getInitTime_flag==1)//第一次来 UTC 数据
        {
            strcpy(lastUpdatedTime,utcTime);//初始化 lastUpdatedTime(只有整数秒)
            getInitTime_flag = 2;
            PPS_isArrived=false;
        }
    }
    if (!isTxInProgress && RingBuffer_Get(&imuDataBuffer, dataToSend)) {
        isTxInProgress = 1; //  设置串口为忙状态
        HAL_UART_Transmit_DMA(&huart5, (uint8_t*)dataToSend, strlen(dataToSend));
    }
  }
  /* USER CODE END 3 */
}
void SystemClock_Config(void)
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
  RCC_OscInitStruct.HSEState = RCC_HSE_ON;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
  RCC_OscInitStruct.PLL.PLLM = 4;
  RCC_OscInitStruct.PLL.PLLN = 96;
  RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2;
  RCC_OscInitStruct.PLL.PLLQ = 4;
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
  {
```

```
    Error_Handler();
  }
  /** Initializes the CPU, AHB and APB buses clocks
  */
  RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                                |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV2;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_1) != HAL_OK)
  {
    Error_Handler();
  }
}
void Error_Handler(void)
{
  /* USER CODE BEGIN Error_Handler_Debug */
  /* User can add his own implementation to report the HAL error return state */
  __disable_irq();
  while (1)
  {
  }
  /* USER CODE END Error_Handler_Debug */
}
#ifdef  USE_FULL_ASSERT
#endif /* USE_FULL_ASSERT */
#include "shared.h"
int isLeapYear(int year) {
    return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
}
int getDaysInMonth(int year, int month) {
    static const int daysInMonth[] = { 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31 };
    if (month == 2 && isLeapYear(year)) {
        return 29;
    }
    return daysInMonth[month - 1];
}
void incrementTime(char *timeStr, int incrementSeconds) {
    int year, month, day, hour, minute, second;
    sscanf(timeStr, "%4d%2d%2d%2d%2d%2d", &year, &month, &day, &hour, &minute, &second);
    second += incrementSeconds;
    while (second >= 60) {
        second -= 60;
        minute++;
        if (minute >= 60) {
            minute = 0;
            hour++;
            if (hour >= 24) {
                hour = 0;
```

```
                day++;
                if (day > getDaysInMonth(year, month)) {
                    day = 1;
                    month++;
                    if (month > 12) {
                        month = 1;
                        year++;
                    }
                }
            }
        }
    }
    sprintf(timeStr, "%04d%02d%02d%02d%02d%02d", year, month, day, hour, minute, second);
}
void IMUDataToString(const IMU_data *data, char *str, size_t maxSize) {
    snprintf(str, maxSize, "%.2f,%.2f,%.2f,%.2f,%.2f,%.2f,%.2f,%.2f,%.2f,%.2f,%.2f,%.2f",
             data->roll, data->pitch, data->yaw,
             data->ax, data->ay, data->az,
             data->gx, data->gy, data->gz,
             data->mx, data->my, data->mz);
}
void Check_LED()
{
    LED0_ON();
    HAL_Delay(300);
    LED0_OFF();
    HAL_Delay(300);
    LED1_ON();
    HAL_Delay(300);
    LED1_OFF();
    HAL_Delay(300);
    LED2_ON();
    HAL_Delay(300);
    LED2_OFF();
    HAL_Delay(300);
}
void Check_BEED()
{
    int i=0;
  while(i<50000)
  {
      HAL_GPIO_WritePin(BEED_GPIO_Port, BEED_Pin, HIGH);
      i++;
  }
  HAL_GPIO_TogglePin(BEED_GPIO_Port, BEED_Pin);//close BEED
}
void RingBuffer_Init(RingBuffer *ringBuffer) {
    ringBuffer->head = 0;
    ringBuffer->tail = 0;
    ringBuffer->count = 0;
```

```
}
int RingBuffer_Put(RingBuffer *ringBuffer, const char *data) {
    if (ringBuffer->count >= BUFFER_SIZE) {
        return 0;  // 缓冲区已满
    }
    strncpy(ringBuffer->buffer[ringBuffer->tail], data, STRING_LENGTH);
    ringBuffer->tail = (ringBuffer->tail + 1) % BUFFER_SIZE;
    ringBuffer->count++;
    return 1;  // 数据添加成功
}
int RingBuffer_Get(RingBuffer *ringBuffer, char *data) {
    if (ringBuffer->count == 0) {
        return 0;  // 缓冲区为空
    }
    strncpy(data, ringBuffer->buffer[ringBuffer->head], STRING_LENGTH);
    ringBuffer->head = (ringBuffer->head + 1) % BUFFER_SIZE;
    ringBuffer->count--;
    return 1;  // 数据获取成功
}
#include "gpio.h"
void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};
  __HAL_RCC_GPIOE_CLK_ENABLE();
  __HAL_RCC_GPIOC_CLK_ENABLE();
  __HAL_RCC_GPIOF_CLK_ENABLE();
  __HAL_RCC_GPIOH_CLK_ENABLE();
  __HAL_RCC_GPIOA_CLK_ENABLE();
  __HAL_RCC_GPIOD_CLK_ENABLE();
  __HAL_RCC_GPIOG_CLK_ENABLE();
  HAL_GPIO_WritePin(GPIOE, GPIO_PIN_4, GPIO_PIN_RESET);
  HAL_GPIO_WritePin(IMUtrigger_GPIO_Port, IMUtrigger_Pin, GPIO_PIN_RESET);
  HAL_GPIO_WritePin(GPIOG, BEED_Pin|LED0_Pin|LED1_Pin|LED2_Pin, GPIO_PIN_RESET);
  GPIO_InitStruct.Pin = GPIO_PIN_4;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(GPIOE, &GPIO_InitStruct);
  GPIO_InitStruct.Pin = KEY1_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_IT_RISING;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  HAL_GPIO_Init(KEY1_GPIO_Port, &GPIO_InitStruct);
  GPIO_InitStruct.Pin = PPS_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_IT_RISING;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  HAL_GPIO_Init(PPS_GPIO_Port, &GPIO_InitStruct);
  GPIO_InitStruct.Pin = IMUtrigger_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
```

```
    HAL_GPIO_Init(IMUtrigger_GPIO_Port, &GPIO_InitStruct);
    GPIO_InitStruct.Pin = BEED_Pin|LED0_Pin|LED1_Pin|LED2_Pin;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOG, &GPIO_InitStruct);
    HAL_NVIC_SetPriority(EXTI9_5_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(EXTI9_5_IRQn);
}
#include "dma.h"
void MX_DMA_Init(void)
{
    __HAL_RCC_DMA1_CLK_ENABLE();
    __HAL_RCC_DMA2_CLK_ENABLE();
    HAL_NVIC_SetPriority(DMA1_Stream0_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(DMA1_Stream0_IRQn);
    HAL_NVIC_SetPriority(DMA1_Stream5_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(DMA1_Stream5_IRQn);
    HAL_NVIC_SetPriority(DMA1_Stream6_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(DMA1_Stream6_IRQn);
    HAL_NVIC_SetPriority(DMA1_Stream7_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(DMA1_Stream7_IRQn);
    HAL_NVIC_SetPriority(DMA2_Stream2_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(DMA2_Stream2_IRQn);
    HAL_NVIC_SetPriority(DMA2_Stream7_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(DMA2_Stream7_IRQn);
}
#include "shared.h"
int isLeapYear(int year) {
        return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
}
int getDaysInMonth(int year, int month) {
        static const int daysInMonth[] = { 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31 };
        if (month == 2 && isLeapYear(year)) {
                return 29;
        }
        return daysInMonth[month - 1];
}
void incrementTime(char *timeStr, int incrementSeconds) {
        int year, month, day, hour, minute, second;
        sscanf(timeStr, "%4d%2d%2d%2d%2d%2d", &year, &month, &day, &hour, &minute, &second);
        second += incrementSeconds;
        while (second >= 60) {
                second -= 60;
                minute++;
                if (minute >= 60) {
                        minute = 0;
                        hour++;
                        if (hour >= 24) {
                                hour = 0;
```

```
                day++;
                if (day > getDaysInMonth(year, month)) {
                    day = 1;
                    month++;
                    if (month > 12) {
                        month = 1;
                        year++;
                    }
                }
            }
        }
    }
    sprintf(timeStr, "%04d%02d%02d%02d%02d%02d", year, month, day, hour, minute, second);
}
void IMUDataToString(const IMU_data *data, char *str, size_t maxSize) {
    snprintf(str, maxSize, "%.2f,%.2f,%.2f,%.2f,%.2f,%.2f,%.2f,%.2f,%.2f,%.2f,%.2f,%.2f",
             data->roll, data->pitch, data->yaw,
             data->ax, data->ay, data->az,
             data->gx, data->gy, data->gz,
             data->mx, data->my, data->mz);
}
void Check_LED()
{
    LED0_ON();
    HAL_Delay(300);
    LED0_OFF();
    HAL_Delay(300);
    LED1_ON();
    HAL_Delay(300);
    LED1_OFF();
    HAL_Delay(300);
    LED2_ON();
    HAL_Delay(300);
    LED2_OFF();
    HAL_Delay(300);
}
void Check_BEED()
{
    int i=0;
  while(i<50000)
  {
      HAL_GPIO_WritePin(BEED_GPIO_Port, BEED_Pin, HIGH);
      i++;
  }
  HAL_GPIO_TogglePin(BEED_GPIO_Port, BEED_Pin);
}
void RingBuffer_Init(RingBuffer *ringBuffer) {
    ringBuffer->head = 0;
    ringBuffer->tail = 0;
    ringBuffer->count = 0;
```

```
}
int RingBuffer_Put(RingBuffer *ringBuffer, const char *data) {
    if (ringBuffer->count >= BUFFER_SIZE) {
        return 0;
    }
    strncpy(ringBuffer->buffer[ringBuffer->tail], data, STRING_LENGTH);
    ringBuffer->tail = (ringBuffer->tail + 1) % BUFFER_SIZE;
    ringBuffer->count++;
    return 1;
}
int RingBuffer_Get(RingBuffer *ringBuffer, char *data) {
    if (ringBuffer->count == 0) {
        return 0;
    }
    strncpy(data, ringBuffer->buffer[ringBuffer->head], STRING_LENGTH);
    ringBuffer->head = (ringBuffer->head + 1) % BUFFER_SIZE;
    ringBuffer->count--;
    return 1;
}
#include "main.h"
#include "dma.h"
#include "tim.h"
#include "usart.h"
#include "gpio.h"
#include <stdbool.h>
#include <string.h>
#include <time.h>
#include <stdio.h>
void SystemClock_Config(void);
uint8_t ss001=0;
int i=0;
bool PPS_isArrived=false;
bool isfirsttime=true;
int   getInitTime_flag=0;
#define IMU_BUFFER_SIZE 170
#define GPS_BUFFER_SIZE 165
#define COMBINED_BUFFER_SIZE 330
char imuBuffer[IMU_BUFFER_SIZE];
char gpsBuffer[GPS_BUFFER_SIZE];
char combinedBuffer[COMBINED_BUFFER_SIZE];
char local_time_ymdsms;
char fisttime;
char current_time_integer;
char lastGpsData[180];
volatile uint8_t gpsDataReady = 0;
volatile uint8_t imuDataReady = 0;
extern uint8_t u1_buffer[85], u2_buffer[150];
extern uint8_t u1_rx_size, u2_rx_size;
volatile int isTxInProgress = 0;
extern DMA_HandleTypeDef hdma_usart1_rx;
```

```
extern DMA_HandleTypeDef hdma_usart1_tx;
extern DMA_HandleTypeDef hdma_usart2_rx;
extern DMA_HandleTypeDef hdma_usart2_tx;
volatile bool pps_flag = false;
volatile int txComplete = 0;
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
        if(GPIO_Pin==PPS_Pin)
        {
            pps_flag = true;
        }
}
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if(htim->Instance == TIM2)
    {
        HAL_GPIO_TogglePin(IMUtrigger_GPIO_Port, IMUtrigger_Pin);
        ss001++;
    }
}
void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart) {
    if (huart == &huart5) {
        isTxInProgress = 0;
        txComplete = 1;
    }
}
int main(void)
{
    IMU_data data={0};
    uint8_t dataimu[500];
    char dataimu_time[256];
    char utcTime[14];
    char lastUpdatedTime[14];
    uint8_t byteArray[sizeof(IMU_data)];
    char timestamp[50];
    RingBuffer imuDataBuffer;
  HAL_Init();
  SystemClock_Config();
  MX_GPIO_Init();
  MX_DMA_Init();
  MX_TIM2_Init();
  MX_USART1_UART_Init();
  MX_USART2_UART_Init();
  MX_UART5_Init();
  Check_LED();
  Check_BEED();
  HAL_UARTEx_ReceiveToIdle_DMA(&huart1, u1_buffer, sizeof(u1_buffer));
  __HAL_DMA_DISABLE_IT(&hdma_usart1_rx, DMA_IT_HT);
  HAL_UARTEx_ReceiveToIdle_DMA(&huart2, u2_buffer, sizeof(u2_buffer));
  __HAL_DMA_DISABLE_IT(&hdma_usart2_rx, DMA_IT_HT);
```

```
strcpy(gpsBuffer, "No GPS data yet");
RingBuffer_Init(&imuDataBuffer);
char dataToSend[STRING_LENGTH];
while (1)
{
        if (pps_flag)
         {
              pps_flag = false;
              HAL_GPIO_TogglePin(LED0_GPIO_Port, LED0_Pin);
              if (isfirsttime && getInitTime_flag==0)
              {
                   isfirsttime = false;
                   getInitTime_flag=1;
                   HAL_TIM_Base_Start_IT(&htim2);
              }
              else {
                   ss001 = 0;
                   __HAL_TIM_SET_COUNTER(&htim2, 0);
                   PPS_isArrived=true;
              }
         }
   if(getInitTime_flag !=1 && PPS_isArrived)
   {
        PPS_isArrived = false;
        incrementTime(lastUpdatedTime, 1);
         long timeDiff = compareTime(lastUpdatedTime,utcTime);
          if ((timeDiff) <1 && (timeDiff) > -1)
          {
          }
          else
          {
               strcpy(lastUpdatedTime, utcTime);
          }
   }
   if (u1_rx_size)
   {
        u1_rx_size=0;
         if (u1_buffer[0] != 0xFA && u1_buffer[1] != 0xFF&&u1_buffer[2] != 0x36) {
             printf("Invalid preamble or bus identifier\n");
        }
        int idx = 4;
        uint32_t raw;
        if (u1_buffer[idx] != 0x20 || u1_buffer[idx+1] != 0x30) {
             printf("Invalid data type identifier for Euler angles\n");
        }
        idx += 2;
        if (u1_buffer[idx] != 0x0C) {
             printf("Invalid length for Euler angles data\n");
        }
        idx++;
```

```
for (int i = 0; i < 3; ++i) {
    raw = (u1_buffer[idx + 4 * i] << 24) |
          (u1_buffer[idx + 1 + 4 * i] << 16) |
          (u1_buffer[idx + 2 + 4 * i] << 8) |
          u1_buffer[idx + 3 + 4 * i];
    float *angle = (float*)&raw;
    switch (i) {
        case 0: data.roll = *angle; break;
        case 1: data.pitch = *angle; break;
        case 2: data.yaw = *angle; break;
    }
}
idx += 12;
if (u1_buffer[idx] != 0x40 || u1_buffer[idx+1] != 0x20) {
    printf("Invalid data type identifier for acceleration\n");
}
idx += 2;
if (u1_buffer[idx] != 0x0C) {
    printf("Invalid length for acceleration data\n");
}
idx++;
for (int i = 0; i < 3; ++i) {
    raw = (u1_buffer[idx + 4 * i] << 24) |
          (u1_buffer[idx + 1 + 4 * i] << 16) |
          (u1_buffer[idx + 2 + 4 * i] << 8) |
          u1_buffer[idx + 3 + 4 * i];
    float *accel = (float*)&raw;
    switch (i) {
        case 0: data.ax = *accel; break;
        case 1: data.ay = *accel; break;
        case 2: data.az = *accel; break;
    }
}
idx += 12;
if (u1_buffer[idx] != 0x40 || u1_buffer[idx+1] != 0x30) {
    printf("Invalid data type identifier for acc_free\n");
}
idx += 2;
if (u1_buffer[idx] != 0x0C) {
    printf("Invalid length for acc_free data\n");
}
idx++;
for (int i = 0; i < 3; ++i) {
    raw = (u1_buffer[idx + 4 * i] << 24) |
          (u1_buffer[idx + 1 + 4 * i] << 16) |
          (u1_buffer[idx + 2 + 4 * i] << 8) |
          u1_buffer[idx + 3 + 4 * i];
    float *Acc_free = (float*)&raw;
    switch (i) {
        case 0: data.ax_free = *Acc_free; break;
```

```
                case 1: data.ay_free = *Acc_free; break;
                case 2: data.az_free = *Acc_free; break;
            }
        }
        idx += 12;
        if (u1_buffer[idx] != 0x80 || u1_buffer[idx+1] != 0x20) {
            printf("Invalid data type identifier for gyro\n");
        }
        idx += 2;
        if (u1_buffer[idx] != 0x0C) {
            printf("Invalid length for gyro data\n");
        }
        idx++;
        for (int i = 0; i < 3; ++i) {
            raw = (u1_buffer[idx + 4 * i] << 24) |
                  (u1_buffer[idx + 1 + 4 * i] << 16) |
                  (u1_buffer[idx + 2 + 4 * i] << 8) |
                  u1_buffer[idx + 3 + 4 * i];
            float *gyro = (float*)&raw;
            switch (i) {
                case 0: data.gx = *gyro; break;
                case 1: data.gy = *gyro; break;
                case 2: data.gz = *gyro; break;
            }
        }
        idx += 12;
        float fractionalSeconds = ss001 * 0.01;
        char fractionalStr[10];
        snprintf(fractionalStr, sizeof(fractionalStr), "%.2f", fractionalSeconds);
        char* decimalPart = fractionalStr + 2;
        snprintf(timestamp, sizeof(timestamp), "%s.%s", lastUpdatedTime, decimalPart);
        char imubuffer[102];
        int len= snprintf(imuBuffer, IMU_BUFFER_SIZE, "%s,%f,%f,%f,%f,%f,%f,%f,%f,%f,%f,%f,%f,%f\r\n",
                timestamp,
                    data.roll, data.pitch, data.yaw,
                    data.ax, data.ay, data.az,
                    data.ax_free, data.ay_free, data.az_free,
                    data.gx, data.gy, data.gz);
        if (RingBuffer_Put(&imuDataBuffer, imuBuffer) == 0) {
        }
        if ((!isTxInProgress && RingBuffer_Get(&imuDataBuffer, dataToSend))|| txComplete) {
            isTxInProgress = 1;
            txComplete = 0;
            HAL_UART_Transmit_DMA(&huart5, (uint8_t*)dataToSend, strlen(dataToSend));
        }
     }
    else if(u2_rx_size)
    {
        if (RingBuffer_Put(&imuDataBuffer, (char*)u2_buffer) == 0) {
        }
```

```
            if ((!isTxInProgress && RingBuffer_Get(&imuDataBuffer, dataToSend))|| txComplete) {
                isTxInProgress = 1;
                txComplete = 0;
                HAL_UART_Transmit_DMA(&huart5, (uint8_t*)dataToSend, strlen(dataToSend));
            }
            memcpy(gpsBuffer, u2_buffer,u2_rx_size);
            u2_rx_size=0;
            extractUTC(gpsBuffer, utcTime);
            if(getInitTime_flag==1)
            {
                strcpy(lastUpdatedTime,utcTime);
                getInitTime_flag = 2;
                PPS_isArrived=false;
            }
        }
        if (!isTxInProgress && RingBuffer_Get(&imuDataBuffer, dataToSend)) {
            isTxInProgress = 1;
            HAL_UART_Transmit_DMA(&huart5, (uint8_t*)dataToSend, strlen(dataToSend));
        }
    }
}
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
    __HAL_RCC_PWR_CLK_ENABLE();
    __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
    RCC_OscInitStruct.HSEState = RCC_HSE_ON;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
    RCC_OscInitStruct.PLL.PLLM = 4;
    RCC_OscInitStruct.PLL.PLLN = 96;
    RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2;
    RCC_OscInitStruct.PLL.PLLQ = 4;
    if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
    {
      Error_Handler();
    }
    RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                                  |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
    RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
    RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV2;
    RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
    RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
    if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_1) != HAL_OK)
    {
      Error_Handler();
    }
}
```

```
void Error_Handler(void)
{
  __disable_irq();
  while (1)
  {
  }
}
#ifdef  USE_FULL_ASSERT
void assert_failed(uint8_t *file, uint32_t line)
{
}
#endif
#include "main.h"
void HAL_MspInit(void)
{
  __HAL_RCC_SYSCFG_CLK_ENABLE();
  __HAL_RCC_PWR_CLK_ENABLE();
}
#include <sys/stat.h>
#include <stdlib.h>
#include <errno.h>
#include <stdio.h>
#include <signal.h>
#include <time.h>
#include <sys/time.h>
#include <sys/times.h>
extern int __io_putchar(int ch) __attribute__((weak));
extern int __io_getchar(void) __attribute__((weak));
char *__env[1] = { 0 };
char **environ = __env;
void initialise_monitor_handles()
{
}
int _getpid(void)
{
  return 1;
}
int _kill(int pid, int sig)
{
  (void)pid;
  (void)sig;
  errno = EINVAL;
  return -1;
}
void _exit (int status)
{
  _kill(status, -1);
  while (1) {}
}
__attribute__((weak)) int _read(int file, char *ptr, int len)
```

```
{
  (void)file;
  int DataIdx;
  for (DataIdx = 0; DataIdx < len; DataIdx++)
  {
    *ptr++ = __io_getchar();
  }
  return len;
}
__attribute__((weak)) int _write(int file, char *ptr, int len)
{
  (void)file;
  int DataIdx;
  for (DataIdx = 0; DataIdx < len; DataIdx++)
  {
    __io_putchar(*ptr++);
  }
  return len;
}
int _close(int file)
{
  (void)file;
  return -1;
}
int _fstat(int file, struct stat *st)
{
  (void)file;
  st->st_mode = S_IFCHR;
  return 0;
}
int _isatty(int file)
{
  (void)file;
  return 1;
}
int _lseek(int file, int ptr, int dir)
{
  (void)file;
  (void)ptr;
  (void)dir;
  return 0;
}
int _open(char *path, int flags, ...)
{
  (void)path;
  (void)flags;
  return -1;
}
int _wait(int *status)
{
```

```
    (void)status;
    errno = ECHILD;
    return -1;
}
int _unlink(char *name)
{
    (void)name;
    errno = ENOENT;
    return -1;
}
int _times(struct tms *buf)
{
    (void)buf;
    return -1;
}
int _stat(char *file, struct stat *st)
{
    (void)file;
    st->st_mode = S_IFCHR;
    return 0;
}
int _link(char *old, char *new)
{
    (void)old;
    (void)new;
    errno = EMLINK;
    return -1;
}
int _fork(void)
{
    errno = EAGAIN;
    return -1;
}
int _execve(char *name, char **argv, char **env)
{
    (void)name;
    (void)argv;
    (void)env;
    errno = ENOMEM;
    return -1;
}
#include <errno.h>
#include <stdint.h>
static uint8_t *__sbrk_heap_end = NULL;
void *_sbrk(ptrdiff_t incr)
{
    extern uint8_t _end;
    extern uint8_t _estack;
    extern uint32_t _Min_Stack_Size;
    const uint32_t stack_limit = (uint32_t)&_estack - (uint32_t)&_Min_Stack_Size;
```

```
    const uint8_t *max_heap = (uint8_t *)stack_limit;
    uint8_t *prev_heap_end;
    if (NULL == __sbrk_heap_end)
    {
      __sbrk_heap_end = &_end;
    }
    if (__sbrk_heap_end + incr > max_heap)
    {
      errno = ENOMEM;
      return (void *)-1;
    }
    prev_heap_end = __sbrk_heap_end;
    __sbrk_heap_end += incr;
    return (void *)prev_heap_end;
}
#include "main.h"
#include "stm32f4xx_it.h"
uint8_t u1_buffer[85] = {0}, u2_buffer[150] = {0};
uint8_t u1_rx_size = 0, u2_rx_size = 0;
extern TIM_HandleTypeDef htim2;
extern DMA_HandleTypeDef hdma_uart5_rx;
extern DMA_HandleTypeDef hdma_uart5_tx;
extern DMA_HandleTypeDef hdma_usart1_rx;
extern DMA_HandleTypeDef hdma_usart1_tx;
extern DMA_HandleTypeDef hdma_usart2_rx;
extern DMA_HandleTypeDef hdma_usart2_tx;
extern UART_HandleTypeDef huart5;
extern UART_HandleTypeDef huart1;
extern UART_HandleTypeDef huart2;
void NMI_Handler(void)
{
    while (1)
    {
    }
}
void HardFault_Handler(void)
{
    while (1)
    {
    }
}
void MemManage_Handler(void)
{
    while (1)
    {
    }
}
void BusFault_Handler(void)
{
    while (1)
```

```
  {
  }
}
void UsageFault_Handler(void)
{
  while (1)
  {
  }
}
void SVC_Handler(void)
{
}
void DebugMon_Handler(void)
{
}
void PendSV_Handler(void)
{
}
void SysTick_Handler(void)
{
  HAL_IncTick();
}
void DMA1_Stream0_IRQHandler(void)
{
  HAL_DMA_IRQHandler(&hdma_uart5_rx);
}
void DMA1_Stream5_IRQHandler(void)
{
  HAL_DMA_IRQHandler(&hdma_usart2_rx);
}
void DMA1_Stream6_IRQHandler(void)
{
  HAL_DMA_IRQHandler(&hdma_usart2_tx);
}
void EXTI9_5_IRQHandler(void)
{
  HAL_GPIO_EXTI_IRQHandler(KEY1_Pin);
  HAL_GPIO_EXTI_IRQHandler(PPS_Pin);
}
void TIM2_IRQHandler(void)
{
  HAL_TIM_IRQHandler(&htim2);
}
void USART1_IRQHandler(void)
{
  HAL_UART_IRQHandler(&huart1);
}
void USART2_IRQHandler(void)
{
  HAL_UART_IRQHandler(&huart2);
```

```
}
void DMA1_Stream7_IRQHandler(void)
{
  HAL_DMA_IRQHandler(&hdma_uart5_tx);
}
void UART5_IRQHandler(void)
{
  HAL_UART_IRQHandler(&huart5);
}
void DMA2_Stream2_IRQHandler(void)
{
  HAL_DMA_IRQHandler(&hdma_usart1_rx);
}
void DMA2_Stream7_IRQHandler(void)
{
  HAL_DMA_IRQHandler(&hdma_usart1_tx);
}
void HAL_UARTEx_RxEventCallback(UART_HandleTypeDef *huart, uint16_t Size)
{
    if(huart == &huart1){
        u1_rx_size = Size;
        HAL_UARTEx_ReceiveToIdle_DMA(&huart1, u1_buffer, sizeof(u1_buffer));
        __HAL_DMA_DISABLE_IT(&hdma_usart1_rx, DMA_IT_HT);
    }else if(huart == &huart2){
        u2_rx_size = Size;
        HAL_UARTEx_ReceiveToIdle_DMA(&huart2, u2_buffer,sizeof(u2_buffer));
        __HAL_DMA_DISABLE_IT(&hdma_usart2_rx, DMA_IT_HT);
    }
}
#include "usart.h"
UART_HandleTypeDef huart5;
UART_HandleTypeDef huart1;
UART_HandleTypeDef huart2;
DMA_HandleTypeDef hdma_uart5_rx;
DMA_HandleTypeDef hdma_uart5_tx;
DMA_HandleTypeDef hdma_usart1_rx;
DMA_HandleTypeDef hdma_usart1_tx;
DMA_HandleTypeDef hdma_usart2_rx;
DMA_HandleTypeDef hdma_usart2_tx;
void MX_UART5_Init(void)
{
  huart5.Instance = UART5;
  huart5.Init.BaudRate = 115200;
  huart5.Init.WordLength = UART_WORDLENGTH_8B;
  huart5.Init.StopBits = UART_STOPBITS_1;
  huart5.Init.Parity = UART_PARITY_NONE;
  huart5.Init.Mode = UART_MODE_TX_RX;
  huart5.Init.HwFlowCtl = UART_HWCONTROL_NONE;
  huart5.Init.OverSampling = UART_OVERSAMPLING_16;
  if (HAL_UART_Init(&huart5) != HAL_OK)
```

```
  {
    Error_Handler();
  }
}
void MX_USART1_UART_Init(void)
{
  huart1.Instance = USART1;
  huart1.Init.BaudRate = 115200;
  huart1.Init.WordLength = UART_WORDLENGTH_8B;
  huart1.Init.StopBits = UART_STOPBITS_1;
  huart1.Init.Parity = UART_PARITY_NONE;
  huart1.Init.Mode = UART_MODE_TX_RX;
  huart1.Init.HwFlowCtl = UART_HWCONTROL_NONE;
  huart1.Init.OverSampling = UART_OVERSAMPLING_16;
  if (HAL_UART_Init(&huart1) != HAL_OK)
  {
    Error_Handler();
  }
}
void MX_USART2_UART_Init(void)
{
  huart2.Instance = USART2;
  huart2.Init.BaudRate = 115200;
  huart2.Init.WordLength = UART_WORDLENGTH_8B;
  huart2.Init.StopBits = UART_STOPBITS_1;
  huart2.Init.Parity = UART_PARITY_NONE;
  huart2.Init.Mode = UART_MODE_TX_RX;
  huart2.Init.HwFlowCtl = UART_HWCONTROL_NONE;
  huart2.Init.OverSampling = UART_OVERSAMPLING_16;
  if (HAL_UART_Init(&huart2) != HAL_OK)
  {
    Error_Handler();
  }
}
void HAL_UART_MspInit(UART_HandleTypeDef* uartHandle)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};
  if(uartHandle->Instance==UART5)
  {
    __HAL_RCC_UART5_CLK_ENABLE();
    __HAL_RCC_GPIOC_CLK_ENABLE();
    __HAL_RCC_GPIOD_CLK_ENABLE();
    GPIO_InitStruct.Pin = GPIO_PIN_12;
    GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
    GPIO_InitStruct.Alternate = GPIO_AF8_UART5;
    HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);
    GPIO_InitStruct.Pin = GPIO_PIN_2;
    GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
```

```
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
    GPIO_InitStruct.Alternate = GPIO_AF8_UART5;
    HAL_GPIO_Init(GPIOD, &GPIO_InitStruct);
    hdma_uart5_rx.Instance = DMA1_Stream0;
    hdma_uart5_rx.Init.Channel = DMA_CHANNEL_4;
    hdma_uart5_rx.Init.Direction = DMA_PERIPH_TO_MEMORY;
    hdma_uart5_rx.Init.PeriphInc = DMA_PINC_DISABLE;
    hdma_uart5_rx.Init.MemInc = DMA_MINC_ENABLE;
    hdma_uart5_rx.Init.PeriphDataAlignment = DMA_PDATAALIGN_BYTE;
    hdma_uart5_rx.Init.MemDataAlignment = DMA_MDATAALIGN_BYTE;
    hdma_uart5_rx.Init.Mode = DMA_NORMAL;
    hdma_uart5_rx.Init.Priority = DMA_PRIORITY_LOW;
    hdma_uart5_rx.Init.FIFOMode = DMA_FIFOMODE_DISABLE;
    if (HAL_DMA_Init(&hdma_uart5_rx) != HAL_OK)
    {
      Error_Handler();
    }
    __HAL_LINKDMA(uartHandle,hdmarx,hdma_uart5_rx);
    hdma_uart5_tx.Instance = DMA1_Stream7;
    hdma_uart5_tx.Init.Channel = DMA_CHANNEL_4;
    hdma_uart5_tx.Init.Direction = DMA_MEMORY_TO_PERIPH;
    hdma_uart5_tx.Init.PeriphInc = DMA_PINC_DISABLE;
    hdma_uart5_tx.Init.MemInc = DMA_MINC_ENABLE;
    hdma_uart5_tx.Init.PeriphDataAlignment = DMA_PDATAALIGN_BYTE;
    hdma_uart5_tx.Init.MemDataAlignment = DMA_MDATAALIGN_BYTE;
    hdma_uart5_tx.Init.Mode = DMA_NORMAL;
    hdma_uart5_tx.Init.Priority = DMA_PRIORITY_MEDIUM;
    hdma_uart5_tx.Init.FIFOMode = DMA_FIFOMODE_DISABLE;
    if (HAL_DMA_Init(&hdma_uart5_tx) != HAL_OK)
    {
      Error_Handler();
    }
    __HAL_LINKDMA(uartHandle,hdmatx,hdma_uart5_tx);
    HAL_NVIC_SetPriority(UART5_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(UART5_IRQn);
  }
  else if(uartHandle->Instance==USART1)
  {
    __HAL_RCC_USART1_CLK_ENABLE();
    __HAL_RCC_GPIOA_CLK_ENABLE();
    GPIO_InitStruct.Pin = GPIO_PIN_9|GPIO_PIN_10;
    GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
    GPIO_InitStruct.Alternate = GPIO_AF7_USART1;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
    hdma_usart1_rx.Instance = DMA2_Stream2;
    hdma_usart1_rx.Init.Channel = DMA_CHANNEL_4;
    hdma_usart1_rx.Init.Direction = DMA_PERIPH_TO_MEMORY;
```

```
    hdma_usart1_rx.Init.PeriphInc = DMA_PINC_DISABLE;
    hdma_usart1_rx.Init.MemInc = DMA_MINC_ENABLE;
    hdma_usart1_rx.Init.PeriphDataAlignment = DMA_PDATAALIGN_BYTE;
    hdma_usart1_rx.Init.MemDataAlignment = DMA_MDATAALIGN_BYTE;
    hdma_usart1_rx.Init.Mode = DMA_NORMAL;
    hdma_usart1_rx.Init.Priority = DMA_PRIORITY_LOW;
    hdma_usart1_rx.Init.FIFOMode = DMA_FIFOMODE_DISABLE;
    if (HAL_DMA_Init(&hdma_usart1_rx) != HAL_OK)
    {
      Error_Handler();
    }
    __HAL_LINKDMA(uartHandle,hdmarx,hdma_usart1_rx);
    hdma_usart1_tx.Instance = DMA2_Stream7;
    hdma_usart1_tx.Init.Channel = DMA_CHANNEL_4;
    hdma_usart1_tx.Init.Direction = DMA_MEMORY_TO_PERIPH;
    hdma_usart1_tx.Init.PeriphInc = DMA_PINC_DISABLE;
    hdma_usart1_tx.Init.MemInc = DMA_MINC_ENABLE;
    hdma_usart1_tx.Init.PeriphDataAlignment = DMA_PDATAALIGN_BYTE;
    hdma_usart1_tx.Init.MemDataAlignment = DMA_MDATAALIGN_BYTE;
    hdma_usart1_tx.Init.Mode = DMA_NORMAL;
    hdma_usart1_tx.Init.Priority = DMA_PRIORITY_LOW;
    hdma_usart1_tx.Init.FIFOMode = DMA_FIFOMODE_DISABLE;
    if (HAL_DMA_Init(&hdma_usart1_tx) != HAL_OK)
    {
      Error_Handler();
    }
    __HAL_LINKDMA(uartHandle,hdmatx,hdma_usart1_tx);
    HAL_NVIC_SetPriority(USART1_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(USART1_IRQn);
  }
  else if(uartHandle->Instance==USART2)
  {
    __HAL_RCC_USART2_CLK_ENABLE();
    __HAL_RCC_GPIOA_CLK_ENABLE();
    GPIO_InitStruct.Pin = GPIO_PIN_2|GPIO_PIN_3;
    GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
    GPIO_InitStruct.Alternate = GPIO_AF7_USART2;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
    hdma_usart2_rx.Instance = DMA1_Stream5;
    hdma_usart2_rx.Init.Channel = DMA_CHANNEL_4;
    hdma_usart2_rx.Init.Direction = DMA_PERIPH_TO_MEMORY;
    hdma_usart2_rx.Init.PeriphInc = DMA_PINC_DISABLE;
    hdma_usart2_rx.Init.MemInc = DMA_MINC_ENABLE;
    hdma_usart2_rx.Init.PeriphDataAlignment = DMA_PDATAALIGN_BYTE;
    hdma_usart2_rx.Init.MemDataAlignment = DMA_MDATAALIGN_BYTE;
    hdma_usart2_rx.Init.Mode = DMA_NORMAL;
    hdma_usart2_rx.Init.Priority = DMA_PRIORITY_LOW;
    hdma_usart2_rx.Init.FIFOMode = DMA_FIFOMODE_DISABLE;
```

```
    if (HAL_DMA_Init(&hdma_usart2_rx) != HAL_OK)
    {
      Error_Handler();
    }
    __HAL_LINKDMA(uartHandle,hdmarx,hdma_usart2_rx);
    hdma_usart2_tx.Instance = DMA1_Stream6;
    hdma_usart2_tx.Init.Channel = DMA_CHANNEL_4;
    hdma_usart2_tx.Init.Direction = DMA_MEMORY_TO_PERIPH;
    hdma_usart2_tx.Init.PeriphInc = DMA_PINC_DISABLE;
    hdma_usart2_tx.Init.MemInc = DMA_MINC_ENABLE;
    hdma_usart2_tx.Init.PeriphDataAlignment = DMA_PDATAALIGN_BYTE;
    hdma_usart2_tx.Init.MemDataAlignment = DMA_MDATAALIGN_BYTE;
    hdma_usart2_tx.Init.Mode = DMA_NORMAL;
    hdma_usart2_tx.Init.Priority = DMA_PRIORITY_LOW;
    hdma_usart2_tx.Init.FIFOMode = DMA_FIFOMODE_DISABLE;
    if (HAL_DMA_Init(&hdma_usart2_tx) != HAL_OK)
    {
      Error_Handler();
    }
    __HAL_LINKDMA(uartHandle,hdmatx,hdma_usart2_tx);
    HAL_NVIC_SetPriority(USART2_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(USART2_IRQn);
  }
}
void HAL_UART_MspDeInit(UART_HandleTypeDef* uartHandle)
{
  if(uartHandle->Instance==UART5)
  {
    __HAL_RCC_UART5_CLK_DISABLE();
    HAL_GPIO_DeInit(GPIOC, GPIO_PIN_12);
    HAL_GPIO_DeInit(GPIOD, GPIO_PIN_2);
    HAL_DMA_DeInit(uartHandle->hdmarx);
    HAL_DMA_DeInit(uartHandle->hdmatx);
    HAL_NVIC_DisableIRQ(UART5_IRQn);
  }
  else if(uartHandle->Instance==USART1)
  {
    __HAL_RCC_USART1_CLK_DISABLE();
    HAL_GPIO_DeInit(GPIOA, GPIO_PIN_9|GPIO_PIN_10);
    HAL_DMA_DeInit(uartHandle->hdmarx);
    HAL_DMA_DeInit(uartHandle->hdmatx);
    HAL_NVIC_DisableIRQ(USART1_IRQn);
  }
  else if(uartHandle->Instance==USART2)
  {
    __HAL_RCC_USART2_CLK_DISABLE();
    HAL_GPIO_DeInit(GPIOA, GPIO_PIN_2|GPIO_PIN_3);
    HAL_DMA_DeInit(uartHandle->hdmarx);
    HAL_DMA_DeInit(uartHandle->hdmatx);
    HAL_NVIC_DisableIRQ(USART2_IRQn);}}
```